Sistem de control al iluminatului inteligent pe arhitectură Zynq

Documentație Tehnică a Proiectului

Student: Bara Andreea

Cuprins

Rezumat **2**

1 Introducere **3**
1.1 Context și motivație . . . 3
1.2 Scopul și obiectivele proiectului . . . 3
1.3 Soluția propusă pe scurt . . . 3
1.4 Structura documentației . . . 4

2 Fundamentare teoretică **5**
2.1 Platforma Zynq și arhitectura PS–PL . . . 5
2.2 Conversia analog–digitală (XADC) . . . 5
2.3 Modulația în lățime de puls (PWM) . . . 5
2.4 Tastatură matricială (scanare pe linii și coloane) . . . 5

3 Proiectare și implementare **7**
3.1 Resurse și semnale . . . 7
3.1.1 Intrări . . . 7
3.1.2 Ieșiri . . . 7
3.2 Arhitectura funcțională a sistemului . . . 7
3.3 Block Design (Vivado) . . . 7
3.4 Componente principale . . . 8
3.5 Implementarea hardware . . . 8
3.5.1 Modul PWM (VHDL) . . . 8
3.5.2 Constrângeri pini (XDC) . . . 8
3.6 Implementarea software (aplicația C în Vitis) . . . 9
3.6.1 Moduri de funcționare . . . 9
3.6.2 Inițializare GPIO (AXI GPIO) . . . 9
3.6.3 Inițializare XADC și citirea senzorului . . . 10
3.6.4 Scanarea tastaturii (Pmod Keypad) . . . 10
3.6.5 Mașina de stări pentru scene . . . 10
3.6.6 Efecte implementate (Party / Breath / Led_breath) . . . 10
3.6.7 Funcția Auto-Off (tasta F) . . . 11
3.6.8 Bucla principală (control AUTO/MANUAL) . . . 11

4 Rezultate experimentale **12**
4.1 Instrumente și mediu de dezvoltare . . . 12
4.2 Procedura de testare . . . 12
4.3 Rezultate observate . . . 12
4.4 Dificultăți și soluții . . . 13

5 Concluzii **14**

Rezumat

Acest document prezintă proiectarea și implementarea unui sistem de control al iluminatului inteligent bazat pe platforma Zynq, utilizând o arhitectură mixtă hardware–software (Vivado + Vitis). Sistemul ajustează automat intensitatea luminii în funcție de iluminarea ambientală, prin citirea unui semnal analogic cu XADC, și permite control manual printr-o tastatură matricială Pmod Keypad. În partea hardware a fost implementat un generator PWM în VHDL, iar în partea software a fost realizată logica de control în limbajul C pe procesorul ARM din PS, incluzând moduri AUTO/MANUAL și scene de iluminare. Rezultatele obținute demonstrează funcționarea corectă a sistemului și evidențiază separarea avantajoasă între decizie (PS) și generarea semnalului de comandă (PL), permițând extinderea ulterioară a funcționalităților.

Capitolul 1

Introducere

1.1 Context și motivație

Iluminatul inteligent este o componentă importantă a sistemelor moderne de automatizare, fiind utilizat pentru eficiență energetică, confort și adaptare la mediul înconjurător. O abordare practică este controlul intensității luminii artificiale în funcție de iluminarea ambientală, completat de posibilitatea intervenției manuale atunci când utilizatorul dorește un comportament diferit (de exemplu, scenarii de iluminare).

1.2 Scopul și obiectivele proiectului

Scopul acestui proiect a fost realizarea unui **sistem de control al iluminatului inteligent** pe platformă Zynq, folosind Vivado pentru partea hardware (PL) și Vitis pentru aplicația software (PS).

Obiectivele principale au fost:

- citirea nivelului de iluminare ambientală prin XADC (canal VAUX, printr-un semnal analogic de tip potențiometru);
- generarea unui semnal PWM în PL pentru controlul intensității a patru LED-uri (LD0–LD3);
- implementarea a două moduri de funcționare:
  - **AUTO** – intensitatea este adaptată automat după valoarea XADC;
  - **MANUAL** – utilizatorul selectează intensitatea/scena cu ajutorul Pmod Keypad;
- controlul unui LED RGB (LRGB) pentru scene de iluminare;
- integrarea și validarea funcțională a sistemului.

1.3 Soluția propusă pe scurt

Soluția utilizează conversia analog–digitală din XADC pentru a obține o valoare numerică (12 biți), care este folosită în mod AUTO pentru a seta factorul de umplere PWM. În mod MANUAL, utilizatorul selectează o scenă sau un nivel de intensitate prin tastatură, iar aplicația C ajustează PWM și/sau culoarea LED-ului RGB.

1.4 Structura documentației

Documentația este organizată astfel: după această introducere urmează o secțiune de fundamentare teoretică, apoi proiectarea și implementarea soluției, rezultatele experimentale, concluziile și bibliografia aferentă tehnologiilor folosite.

Capitolul 2

Fundamentare teoretică

2.1 Platforma Zynq și arhitectura PS–PL

Platformele Zynq integrează într-un singur circuit două părți principale: **PS (Processing System)**, bazat pe procesoare ARM, și **PL (Programmable Logic)**, de tip FPGA. Această arhitectură permite implementarea unor funcții de control și decizie în software (PS) și a unor funcții de timp real / semnale digitale în hardware (PL), cu comunicație prin interfețe standard (de exemplu AXI). Prin această separare, se obțin flexibilitate în dezvoltare și performanță în generarea semnalelor.

2.2 Conversia analog–digitală (XADC)

Pentru a măsura un semnal analogic este necesară conversia analog–digitală. XADC (Xilinx Analog-to-Digital Converter) permite citirea unor canale analogice (de tip VAUX) și furnizează o valoare digitală. În mod tipic, valoarea brută este disponibilă într-un format mai larg, iar pentru aplicații de control simplu se poate utiliza componenta relevantă pe 12 biți. Citirea repetată în mod continuu (conversie continuă) permite actualizarea periodică a valorii măsurate.

2.3 Modulația în lățime de puls (PWM)

PWM este o tehnică de control a puterii medii livrate către o sarcină digitală prin comutarea rapidă între 0 și 1. Un semnal PWM este caracterizat de **perioadă** și **factor de umplere** (duty-cycle). Puterea medie percepută (de exemplu luminozitatea unui LED) este aproximativ proporțională cu factorul de umplere:

$$D = \frac{T_{ON}}{T_{PER}}$$

În implementări digitale, PWM se realizează uzual prin compararea unui contor cu o valoare de referință (`duty`). Dacă `cnt < duty` atunci ieșirea este 1, altfel 0. Pentru un `duty` pe 12 biți, intervalul este 0–4095, unde 0 înseamnă LED stins, iar 4095 înseamnă intensitate maximă (aproape 100%).

2.4 Tastatură matricială (scanare pe linii și coloane)

O tastatură matricială 4x4 utilizează 4 linii și 4 coloane. Apăsarea unei taste conectează o linie cu o coloană. Scanarea se face activând pe rând fiecare coloană (de exemplu setând-o LOW)

și citind liniile; dacă o linie devine LOW, atunci tasta corespunzătoare (linie, coloană) este apăsată. Indexul poate fi determinat, de exemplu, prin:

$$index = row \times 4 + col$$

Ulterior, indexul se mapează la simbolul dorit (0–9, A–F etc.).

Capitolul 3

Proiectare și implementare

3.1 Resurse și semnale

3.1.1 Intrări

- **Senzor de lumină**: semnal analogic (potențiometru) citit prin XADC pe un canal VAUX (ex.: VAUX14).
- **Tastatură**: Pmod KYPD (matrice 4x4).
- **SW0**: 0 – Mod AUTO, 1 – Mod MANUAL.

3.1.2 Ieșiri

- **LD0–LD3**: controlate prin semnal PWM (intensitate variabilă).
- **LRGB**: control pentru scene (culoare/intensitate în funcție de scenă).

3.2 Arhitectura funcțională a sistemului

Logica de funcționare a sistemului a fost următoarea:

- sistemul a măsurat lumina ambientală prin XADC;
- valoarea măsurată a fost utilizată pentru a decide intensitatea PWM;
- dacă lumina ambientală a fost scăzută, intensitatea luminii artificiale a crescut;
- dacă lumina ambientală a fost ridicată, intensitatea a scăzut până la oprire;
- utilizatorul a putut trece oricând în mod MANUAL (SW0=1) și a putut seta intensitatea/scena din tastatură.

3.3 Block Design (Vivado)

Figura 3.1 prezintă integrarea componentelor în mediul de dezvoltare Vivado.

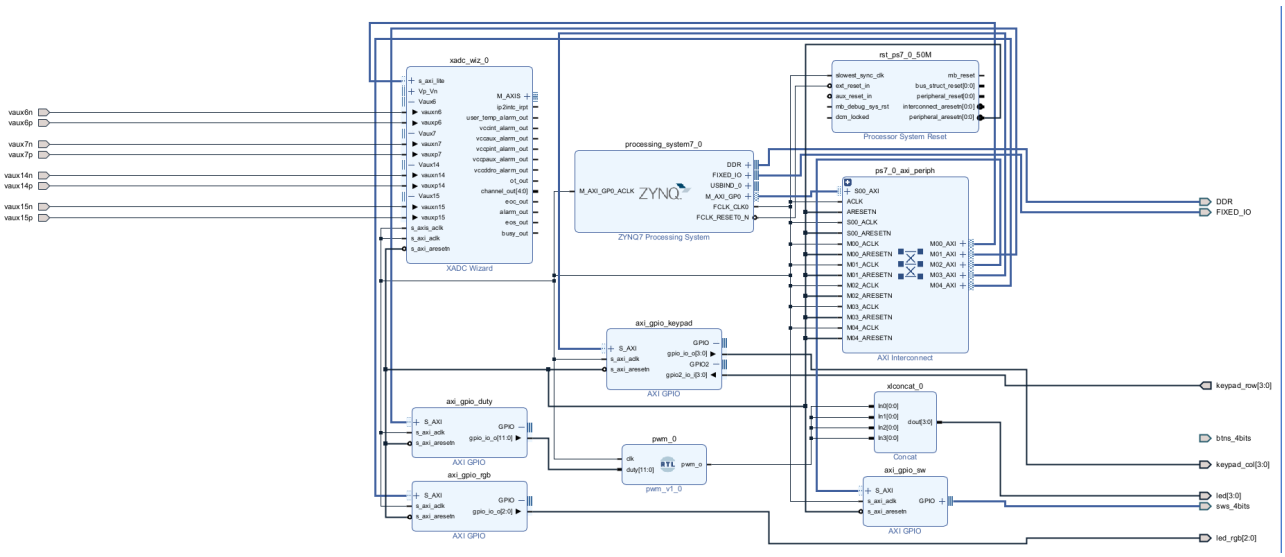

Figura 3.1: Block Design – integrarea PS7, XADC, AXI GPIO și PWM

3.4 Componente principale

Sistemul a fost construit din următoarele componente (în Block Design):

- **processing_system7_0**: interfață PS–PL, clock/reset, DDR;
- **xadc_wiz_0**: citire canale analogice VAUX;
- **ps7_0_axi_periph**: interconect AXI;
- **rst_ps7_0_50M**: reset sincron pentru domeniul de ceas utilizat;
- **axi_gpio_duty**: scriere duty (12 biți) către IP-ul PWM;
- **pwm_0**: generare semnal PWM în PL;
- **axi_gpio_rgb**: control LED RGB;
- **axi_gpio_keypad**: citire tastatură (scanare matricială);
- **axi_gpio_sw**: citire switch-uri / semnale de control.

3.5 Implementarea hardware

3.5.1 Modul PWM (VHDL)

Semnalul PWM a fost generat prin compararea unui contor cu valoarea `duty`. Ieșirea a fost `1` cât timp `cnt < duty`, obținându-se un factor de umplere proporțional cu `duty` (0–4095 pentru 12 biți). Modulul a fost descris în limbajul VHDL și integrat ca bloc custom IP în proiectul general.

3.5.2 Constrângeri pini (XDC)

Fișierul de constrângeri XDC a definit maparea semnalelor pe pini (LED-uri, RGB, keypad etc.). Aceste constrângeri au asigurat maparea corectă a semnalelor logice la pinii fizici ai platformei de dezvoltare.

3.6 Implementarea software (aplicația C în Vitis)

3.6.1 Moduri de funcționare

AUTO

În modul AUTO (SW0=0), sistemul a ajustat singur intensitatea în funcție de lumina ambientală:

- lumină scăzută ⇒ creștere intensitate (duty mai mare);
- lumină ridicată ⇒ scădere intensitate sau oprire (duty mic/0).

MANUAL

În modul MANUAL (SW0=1), utilizatorul a ales intensitatea sau scena folosind tastatura:

- **0**: mod empty, toate LED-urile oprite
- **1**: intensitate mică, 20%
- **2**: intensitate medie, 55%
- **3**: intensitate maximă, 90%
- **4**: scena relax: lumină ambientală pe LD0–LD3 + LRGB albastru low intensity
- **5**: scena party: LRGB își schimbă culoarea la un interval de timp, iar LD0–LD3 sunt stinse
- **7**: scena led_breath: cele 4 LED-uri (LD0–LD3) au efect *breathing*
- **F**: scena auto_off: LED-urile se sting treptat indiferent de scena activă

3.6.2 Inițializare GPIO (AXI GPIO)

În `main()` au fost inițializate instanțele `XGpio` pentru:

- `inputPWM` – scrierea valorii `duty` către IP-ul PWM;
- `inputSw` – citirea switch-urilor (SW0);
- `keypad` – matricea tastaturii (coloane ca output, rânduri ca input);
- `rgb` – control pentru LED RGB.

Direcțiile au fost setate astfel:

- PWM: output (`0x0000`);
- Switch: input (`0xF`);
- Keypad CH1 (coloane): output, Keypad CH2 (rânduri): input;
- RGB: output.

3.6.3 Inițializare XADC și citirea senzorului

XADC a fost configurat în mod de conversie continuă (`CONTINPASS`), iar canalele VAUX au fost activate. În bucla principală s-a citit un canal (de ex. VAUX14), iar valoarea brută a fost transformată în 12 biți prin:

- shift la dreapta cu 4 biți (ignorarea biților LSB nefolosiți);
- mascarea pe 12 biți (`0x0FFF`).

Această valoare a devenit direct `duty_value` în mod AUTO.

3.6.4 Scanarea tastaturii (Pmod Keypad)

Tastatura a fost scanată prin activarea pe rând a fiecărei coloane (un singur bit LOW), apoi citirea rândurilor. Algoritmul a fost:

1. activarea coloanei curente (CH1);
2. citirea rândurilor (CH2);
3. dacă un rând a devenit LOW, înseamnă că a fost apăsată o tastă;
4. calcularea indexului tastelor (0..15) ca `row * 4 + col`.

Indexul a fost mapat la cifre/scenă cu o funcție dedicată.

3.6.5 Mașina de stări pentru scene

Aplicația a folosit un `enum` pentru scene:

- `SCENE_EMPTY` – LED-uri oprite;
- `SCENE_LEVEL` – intensitate fixă (20%, 55%, 90%);
- `SCENE_RELAX` – efect de tip breathing + RGB (albastru) pentru ambient;
- `SCENE_PARTY` – RGB își schimbă culoarea periodic;
- `SCENE_LED_BREATH` – breathing pe LD0–LD3 (duty urcă/coboară);
- `SCENE_AUTO_OFF` – stingere treptată în timp fix.

O funcție specifică a setat scena curentă și a inițializat variabilele interne (duty, direcții, contoare).

3.6.6 Efecte implementate (Party / Breath / Led_breath)

Party: s-a schimbat culoarea RGB după un număr fix de iterații, utilizând un contor și o listă de culori.

Breath (Relax): s-a modificat gradual intensitatea între un minim și un maxim, schimbându-se culoarea RGB la limite.

Led_breath: s-a calculat un duty care a urcat și a coborât între 0 și 4095, realizând efectul de breathing pe LD0–LD3.

3.6.7 Funcția Auto-Off (tasta F)

La detectarea tastei **F**, s-a pornit secvența `auto_off`. Apoi, cât timp scena curentă a fost setată pe stingere, logica a urmat pașii:

- scăderea valorii curentului gradual astfel încât să ajungă **exact la 0 în 10 secunde**;
- scrierea duty-ului rezultat către PWM la fiecare iterație;
- la final, sistemul a fost setat pe oprit complet.

3.6.8 Bucla principală (control AUTO/MANUAL)

În fiecare iterație:

- s-a citit statusul switch-ului de mod;
- în mod AUTO: intensitatea a fost suprascrisă de senzorul analogic;
- în mod MANUAL: s-a scanat tastatura, s-a aplicat scena, apoi s-a calculat valoarea de ieșire în funcție de scena curentă;
- s-a actualizat semnalul către hardware.

Capitolul 4

Rezultate experimentale

4.1 Instrumente și mediu de dezvoltare

Pentru realizarea proiectului au fost utilizate următoarele:

- Vivado (pentru Block Design, integrare IP, generare bitstream);
- Vitis (pentru aplicația C pe PS);
- limbaje: VHDL (PL) și C (PS);
- implementare pe platformă Zynq (cu perifericele LED și Pmod Keypad).

4.2 Procedura de testare

Testarea s-a realizat prin parcurgerea următoarelor scenarii:

- **Test mod AUTO**: variarea potențiometrului (iluminare) și observarea modificării luminozității LED-urilor LD0–LD3.
- **Test mod MANUAL**: apăsarea tastelor 0/1/2/3/4/5/7/F și observarea comportamentului LED-urilor și RGB.
- **Test Auto-Off**: selectarea unei scene cu intensitate non-zero, apoi apăsarea tastei F și verificarea stingerii treptate până la 0 în aproximativ 10 secunde.

4.3 Rezultate observate

- În modul AUTO, sistemul a crescut intensitatea PWM la scăderea iluminării (valoare XADC mai mică) și a redus intensitatea la creșterea iluminării.
- În modul MANUAL, fiecare tastă a activat corect nivelul/scena asociată, iar LED RGB a indicat scenele care necesitau efecte color.
- Efectele (party/breath/led_breath) au fost stabile, iar tranzițiile au fost continue datorită actualizării periodice a duty-cycle-ului.
- Funcția Auto-Off a redus treptat duty-ul până la 0, apoi a trecut sistemul în starea de LED-uri oprite.

4.4 Dificultăți și soluții

- **Scanarea tastaturii**: a fost necesară o mapare corectă coloană–rând și o temporizare minimă pentru stabilizarea citirii.
- **Conversia XADC**: a fost necesară prelucrarea rezultatului brut (shift și mască) pentru a obține o valoare stabilă pe 12 biți, potrivită pentru PWM.
- **Auto-Off exact 10 secunde**: a fost necesară corelarea scăderii duty-ului cu perioada de actualizare în buclă, astfel încât stingerea să fie uniformă.

Capitolul 5

Concluzii

Proiectul a realizat un sistem de control al iluminatului inteligent pe arhitectură Zynq, cu mod automat bazat pe citirea XADC și mod manual cu scene selectate dintr-o tastatură matricială. Separarea PS (decizie și logică de control) / PL (PWM și interfațare hardware) a oferit un comportament stabil și o extindere ușoară a funcționalităților.

Sistemul confirmă utilitatea arhitecturilor de tip System-on-Chip (SoC) pentru aplicații de automatizare unde este necesară atât putere de calcul secvențială (procesor ARM), cât și generare de semnale de mare viteză (FPGA).

Dezvoltări viitoare posibile:

- filtrare software a valorii XADC (medie mobilă) pentru o stabilitate mai bună;
- introducerea unor praguri definite pentru AUTO (histerezis) pentru evitarea oscilațiilor;
- adăugarea unor scene suplimentare și configurare dinamică (de exemplu prin conectivitate wireless sau UART).

Bibliografie

[1] J. L. Hennessy, D. A. Patterson, *Organizarea și proiectarea calculatoarelor – interfața hardware/software*, Editura All Educational, București, 2002.

[2] Xilinx, *XADC User Guide (UG480)*, Documentație tehnică Xilinx.

[3] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, Documentație tehnică Xilinx.

[4] A. Oppenheim, R. Schafer, *Discrete-Time Signal Processing.*